UCLA Samueli School of Engineering

CS 35L Software Construction

Lecture 4 – Version Control & Git

Tobias Dürschmid
Assistant Teaching Professor
Computer Science Department

Version Control Definition

Version control (also known as *revision control*, *source control*, and *source code management*) is the software engineering practice of **controlling**, **organizing**, and **tracking different versions** in the history of **computer files**; primarily source code text files, but generally any type of file (usually works best with text files)

Manual Version Control is Cumbersome and Error-Prone

Homework.txt

Homework_2.txt

Homework_3.txt

Homework_submitted.txt

Homework_submitted_fixed.txt

Homework_submitted_fixed_also_typos_fixed.txt

Homework_submitted_fixed_also_typos_fixed_really_finished.txt

Why Do We Need Version Control?

Collaboration

Enables **multiple developers** to work **concurrently** on the same project **without overwriting each other's changes**.

Change Tracking

Developers can **see what has changed** since they last worked on a particular piece of code.

Traceability

Provides a summary of the history of every **modification**, **who** made it, **when**, and (ideally) **why**.

Why Do We Need Version Control?

Reversion / Rollback

Allows easy **reversion to previous versions** if **errors** are introduced or **features need to be rolled back**

Parallel Development

Supports **parallel development** of features or bug fixes without affecting the main codebase. This allows developers to experiment in isolation.

Code Reviews

Code can be **reviewed by other developers before** it is **integrated** into the main system.

Version Control System Definition

A *version control system* is a **software tool** that **automates version control.**

Examples:

By By far the most common VCS

The Software Engineer's Time Stone

- **Free and open-source** distributed version control system
- Created by Linus Torvalds (Linux creator) (since 2005)
- Used **by almost all open source systems**
- Used **by many companies**
- Has powerful algorithms for combining source code changes from different versions

See https://git-scm.com/

Video Tutorial available here: https://www.youtube.com/watch?v=8JJ101D3knE&

Git Keeps the Files you See and Edit Separate from the Recorded History (in files stored in the hidden .git folder)

Working Tree

(aka. “working copy”) The current version of your files as you see them in your code editor

A complete history of all changes to the project (on your machine!)

Local Repository

Git Keeps the Files you See and Edit Separate from the Recorded History (in files stored in the hidden .git folder)

Working Tree

File 1

File 2

(aka. "working copy") The current version of your files as you see them in your code editor

A complete history of all changes to the project (on your machine!)

Local Repository

Local Changes must be explicitly added to the Staging Area

See https://git-scm.com/about/staging-area

A selection of changes that are to be committed to your local repo. Think of it like a "draft" of a version that you put in your shopping cart

Working Tree

File 1'

File 2'

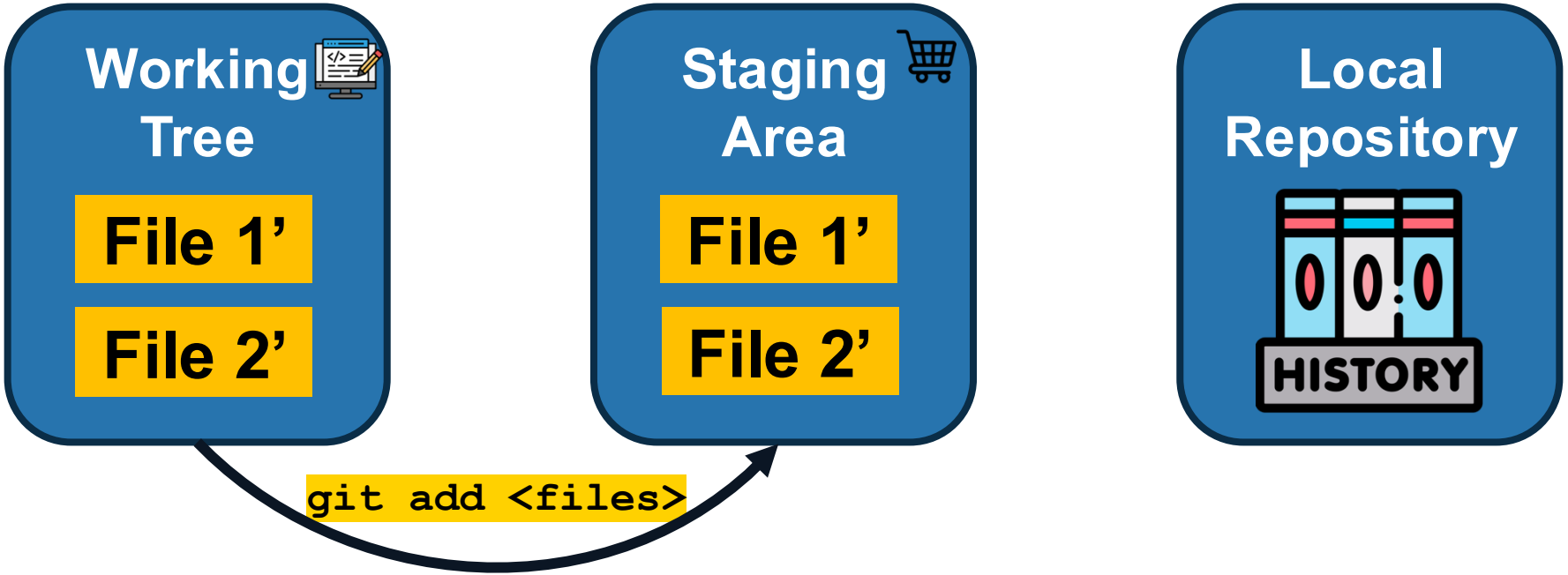
Staging Area

Local Repository

Local Changes must be explicitly added to the Staging Area via `git add`

See https://git-scm.com/docs/git-add

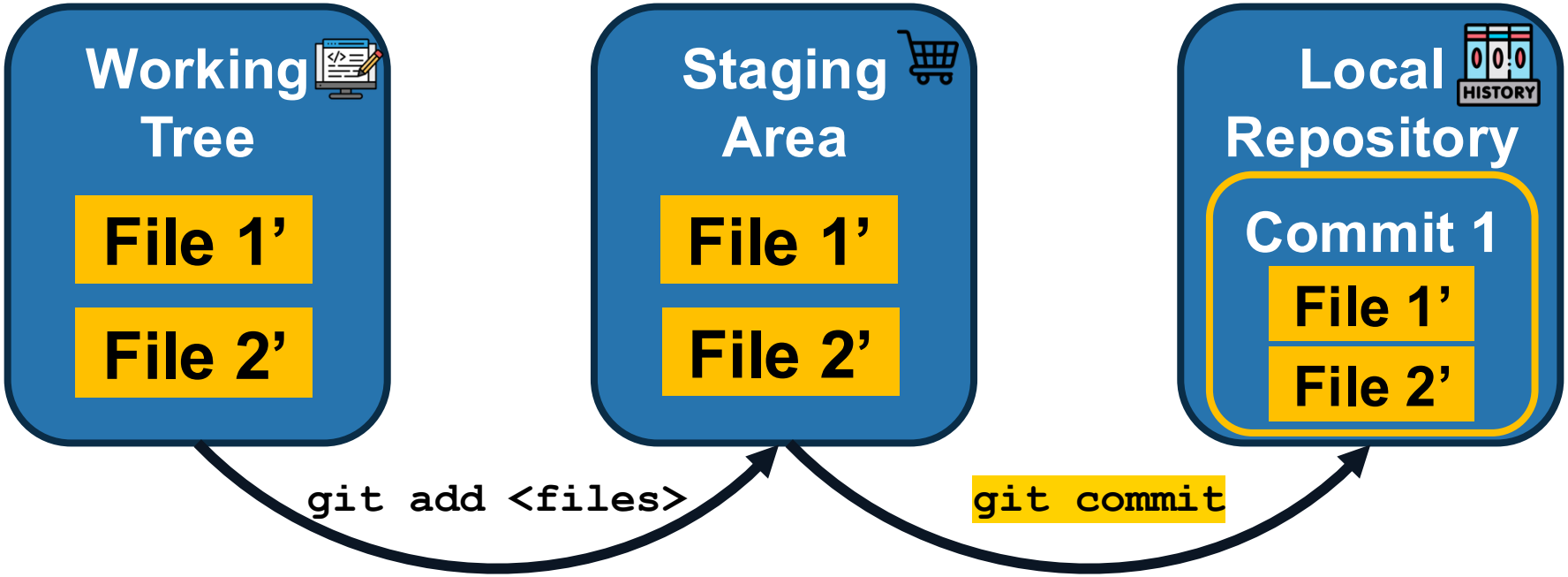
`git add *.py` adds all .py files; `git add -A` adds all files in the repo
`git add <dir>` adds all files recursively in the directory



Create a Named Version of The Content in the Staging Area via: `git commit`

See https://git-scm.com/docs/git-commit

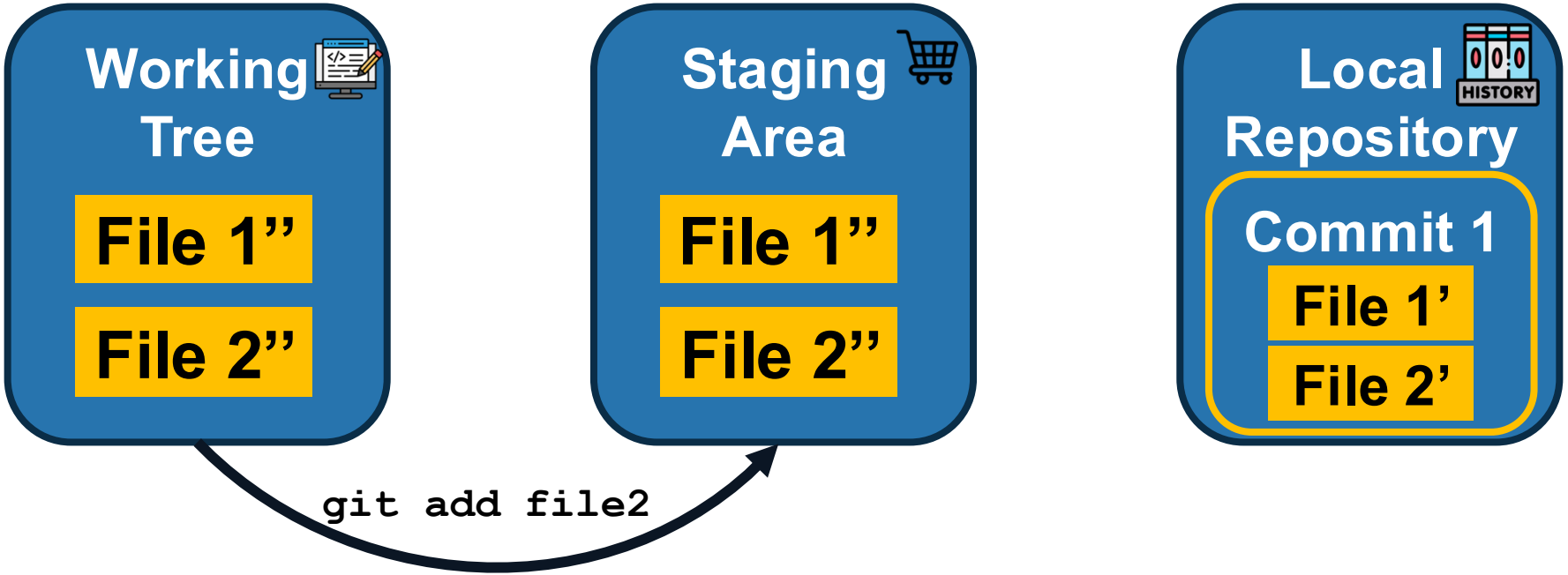
`git commit -m "<description of the change>"`
lets you add the commit message directly in the command



See https://git-scm.com/docs/git-commit

`git commit` Clears the Staging Area

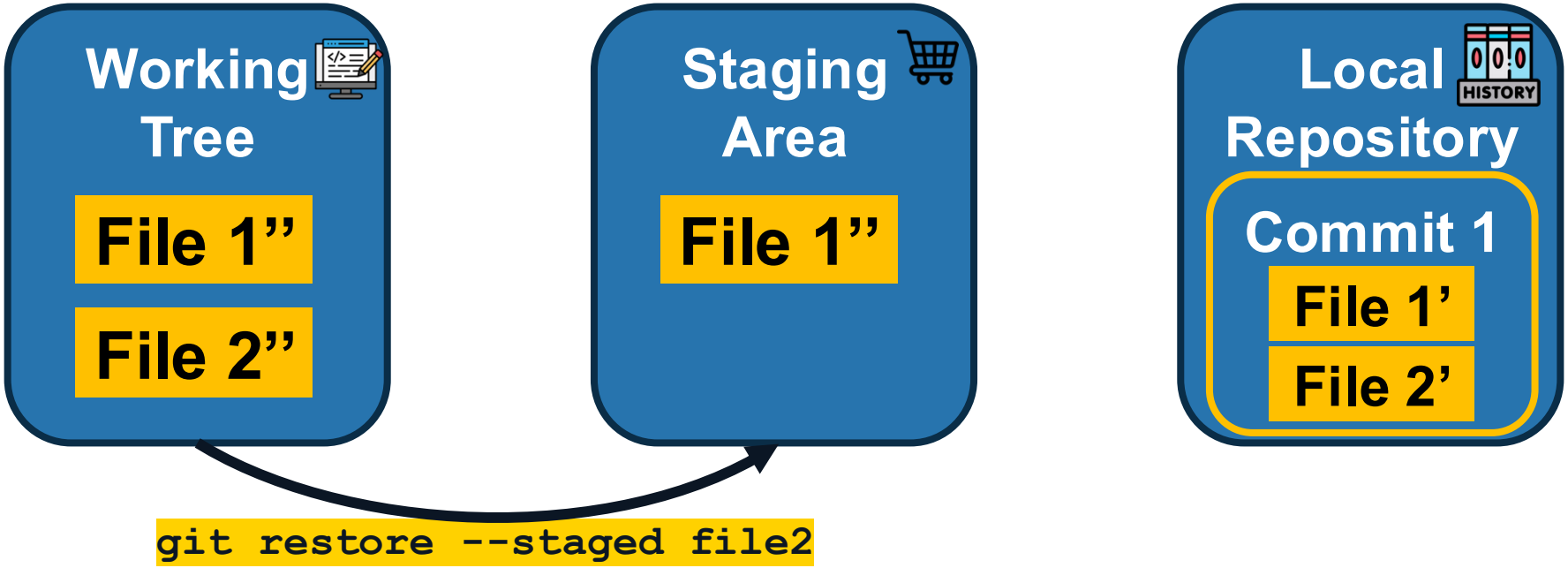
`git commit` Clears the Staging Area



Unstage Local Changes via `git restore --staged`

See https://git-scm.com/docs/git-restore

Undo for `git add`



What happens when we commit after changing both files again? Talk to your neighbor(s)!

A: Commit 2 would contain File 1” (not ”’ because only the previous version was staged) and File 2’ (since we previously unstaged it)

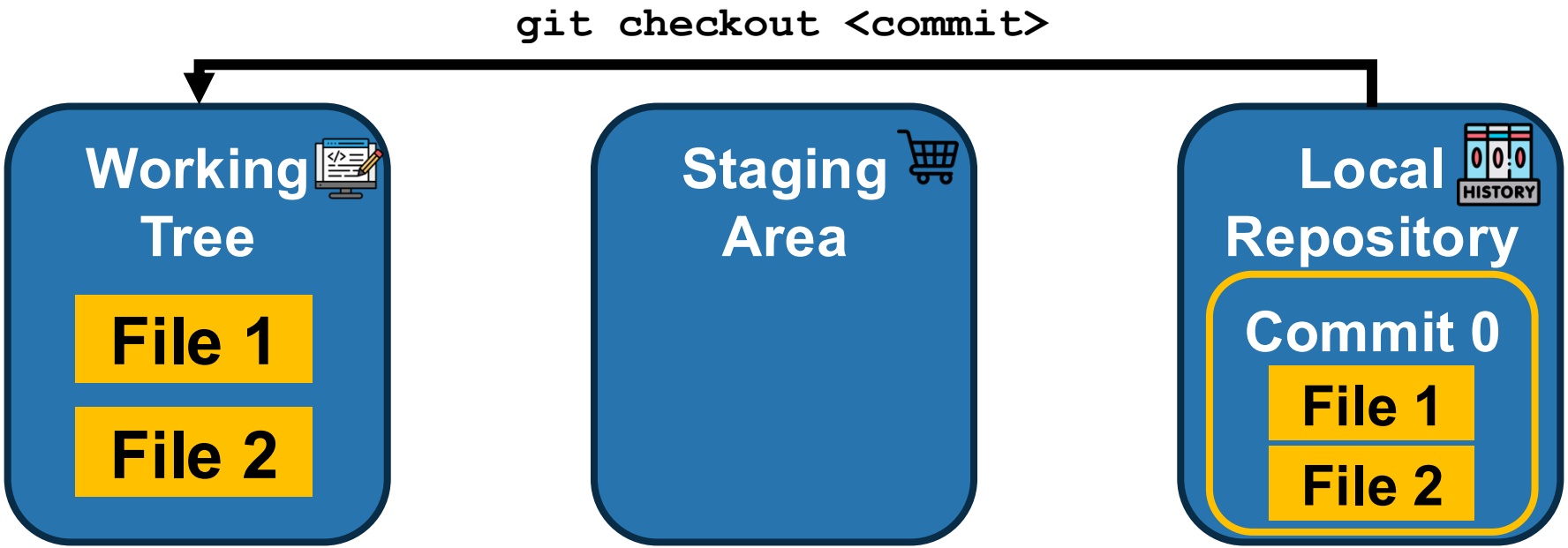
Undo all Tracked Changes in Your Working Tree via `git reset --hard`

See https://git-scm.com/docs/git-reset

Will not change files that are not tracked (i.e., have never been added to your repo)

See https://git-scm.com/docs/git-checkout

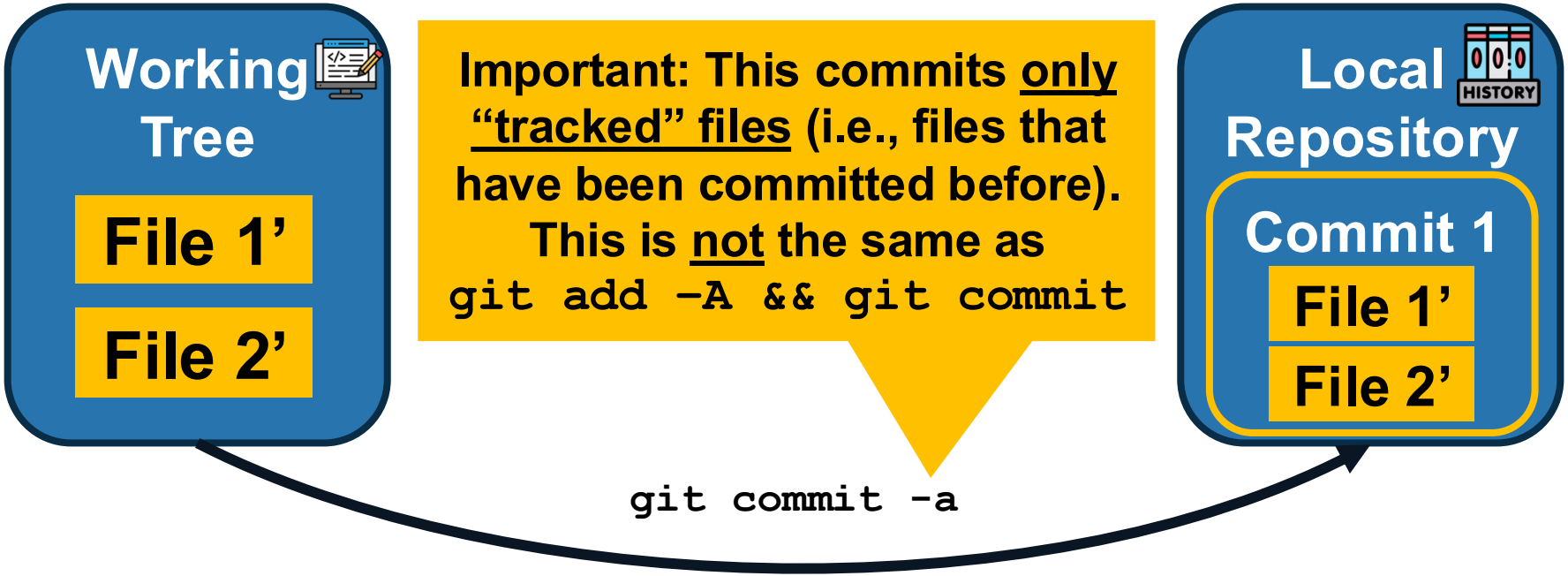
Restore a previous version via `git checkout`



See https://git-scm.com/docs/git-commit

Skipping the Staging Area: `git commit -a`

`git commit -am "<description of the change>"`
lets you add the commit message directly in the command



`.gitignore` keeps your repo clean from temporary files or build artefacts

- Each directory in a git repo can have a file named `.gitignore` that specifies which files should be ignored (i.e., should not be tracked)

```
# You can specify directories to be ignored
build/

# You can specify file types to be ignored
*.log

# You can make exceptions to ignore rules
*.pdf #ignores all pdf files
!/images/**/*.pdf #adds pdf files that are within the images folder
```

See https://git-scm.com/docs/gitignore

View your staged and unstaged changes via `git status`

See https://git-scm.com/docs/git-status

Terminal

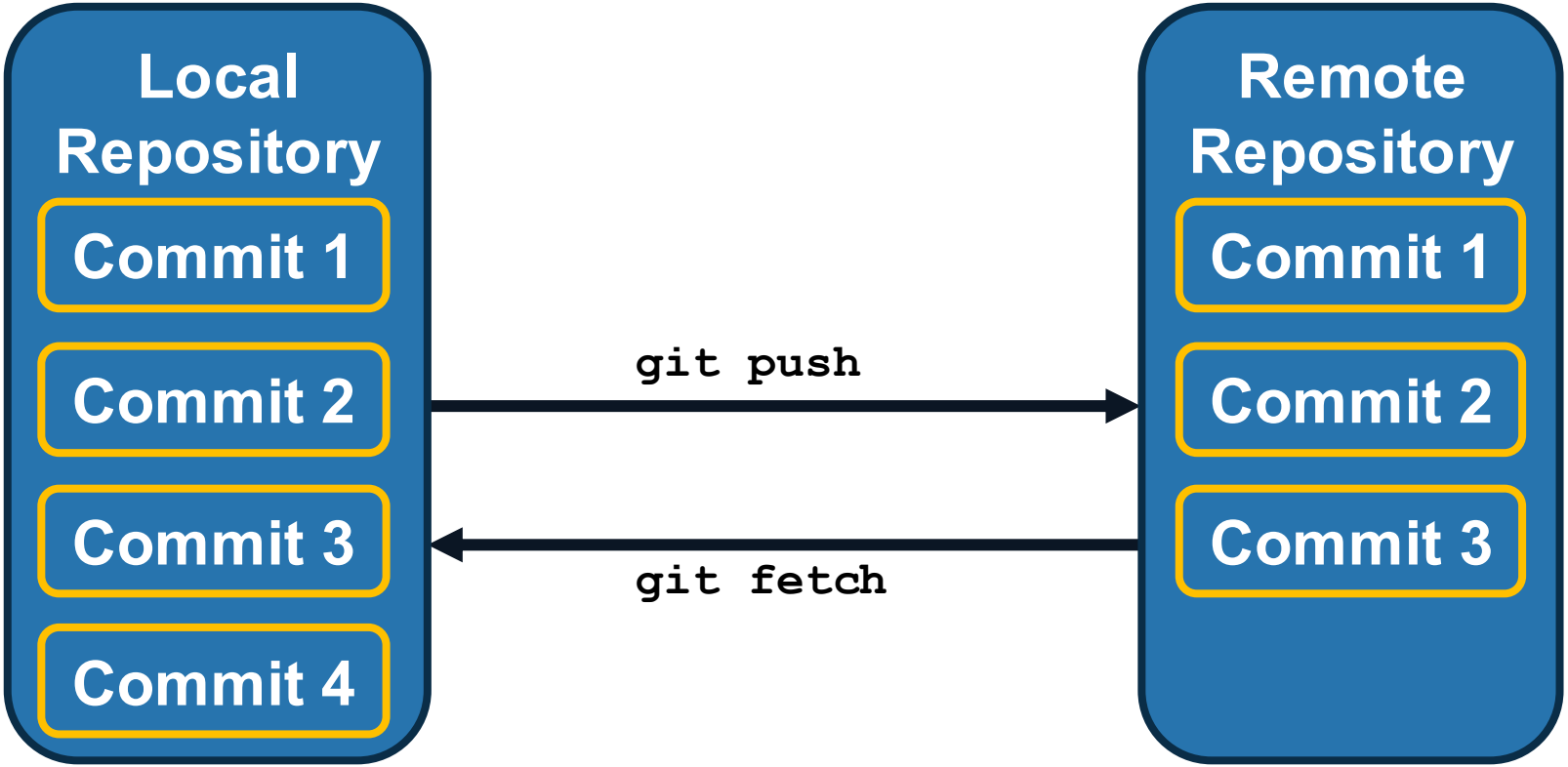
```
$git status
On branch main. Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
new file:   file1

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
deleted:    file1
```

Staging Area

What has happened here???

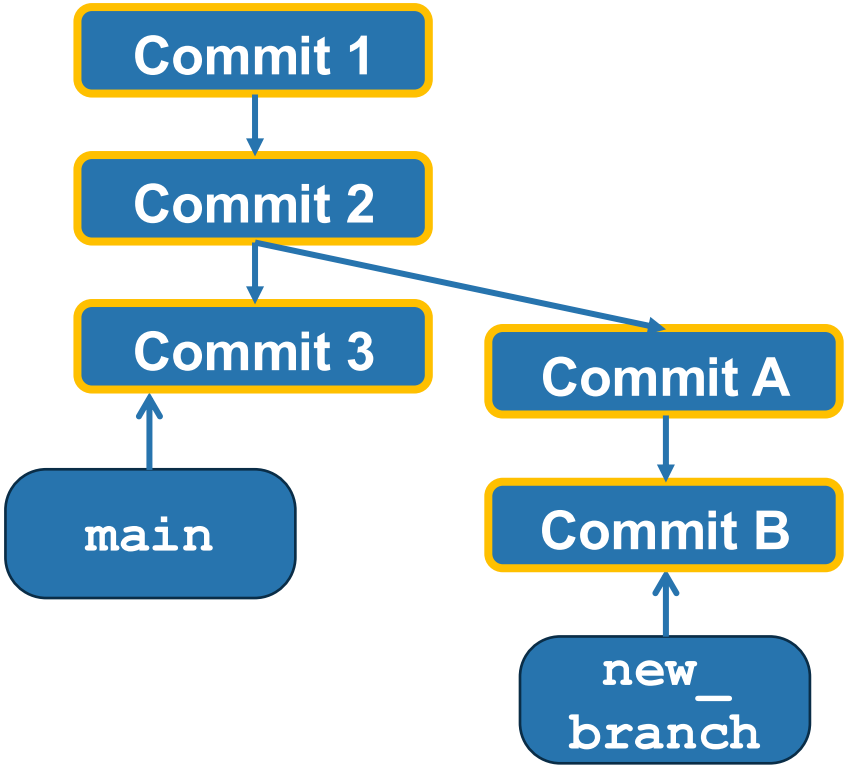
Other changes in the working tree

Communicate with the Remote Repo via `git push` and `git fetch`



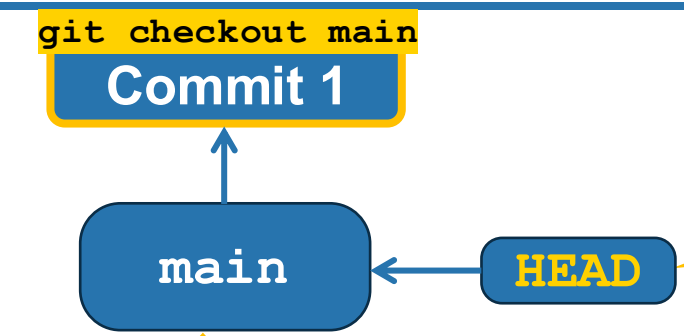
Branches allow Parallel Development of Different Features in Isolation

See https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell



See https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Switch to an existing branch with git checkout

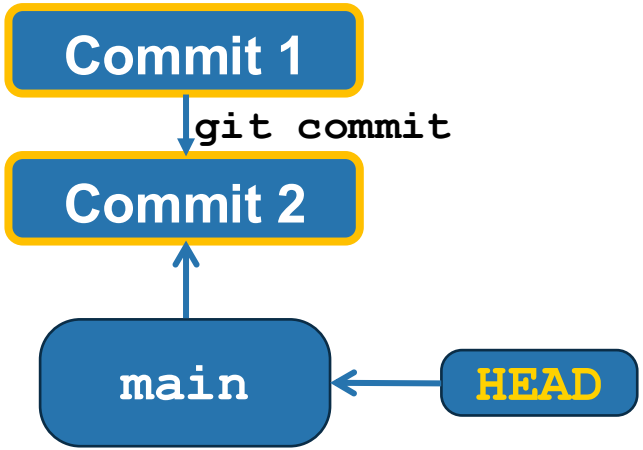


HEAD is pointing to the branch / commit you are currently on

Branches are pointers to a commit

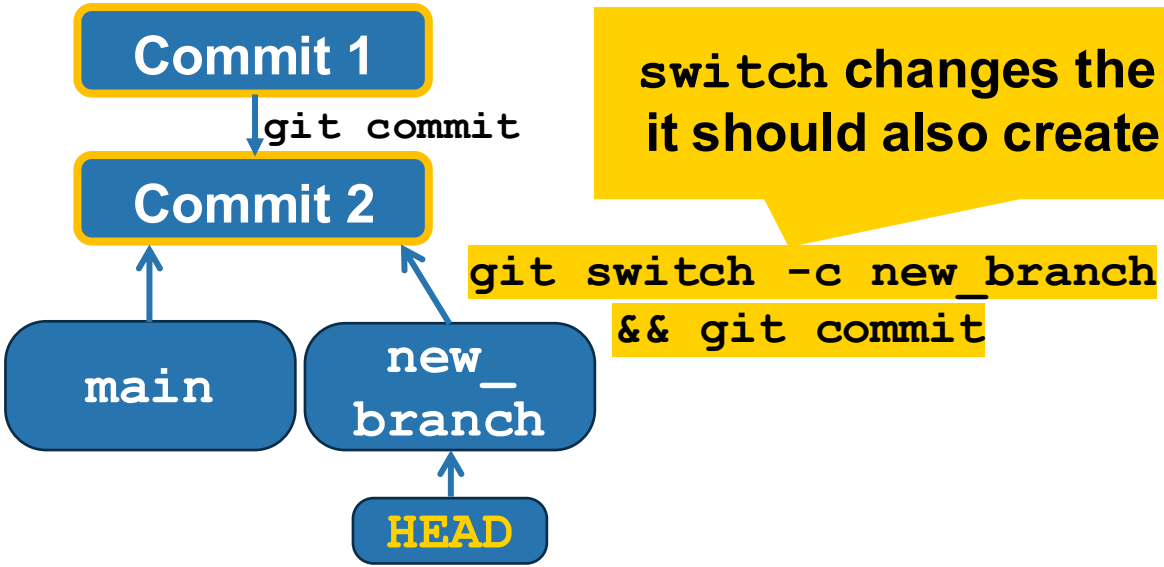
See https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Create a new branch



See https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Create a new branch with `git switch -c`

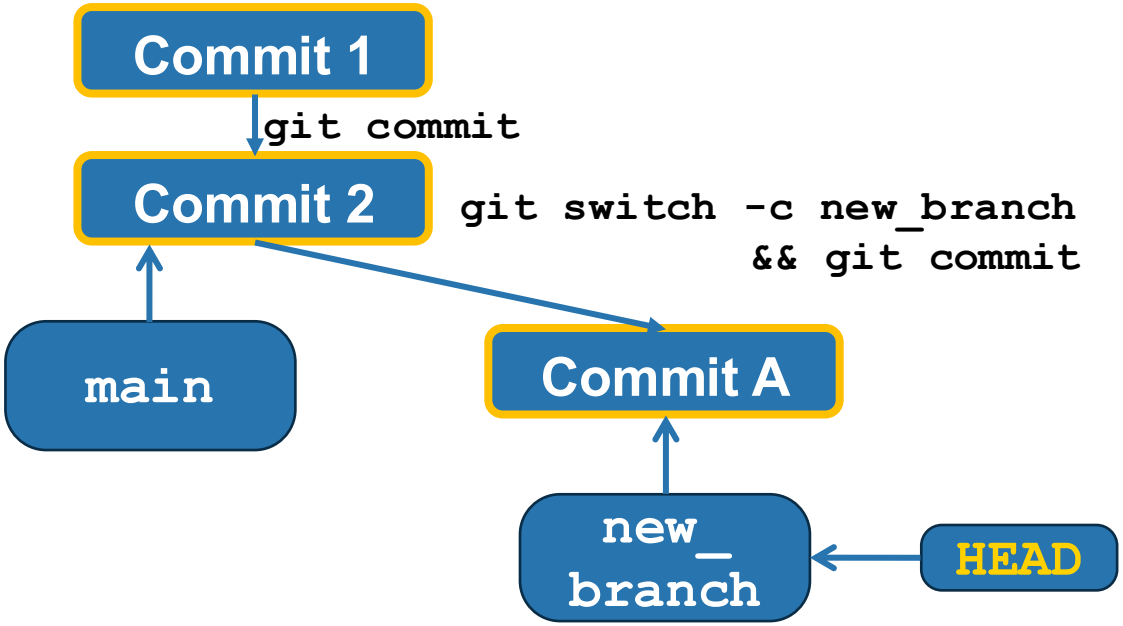


`switch` changes the branch. `-c` says it should also create the new branch

```
git switch -c new_branch
&& git commit
```

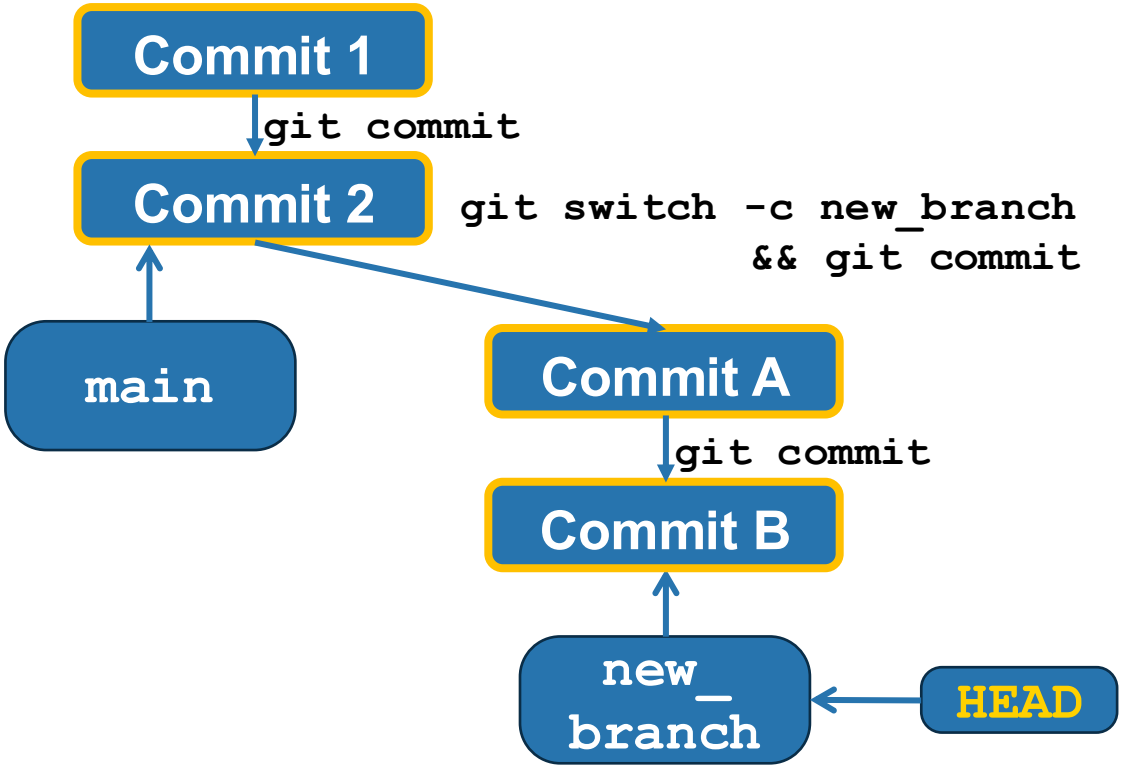
See https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Create a new branch with `git switch -c`



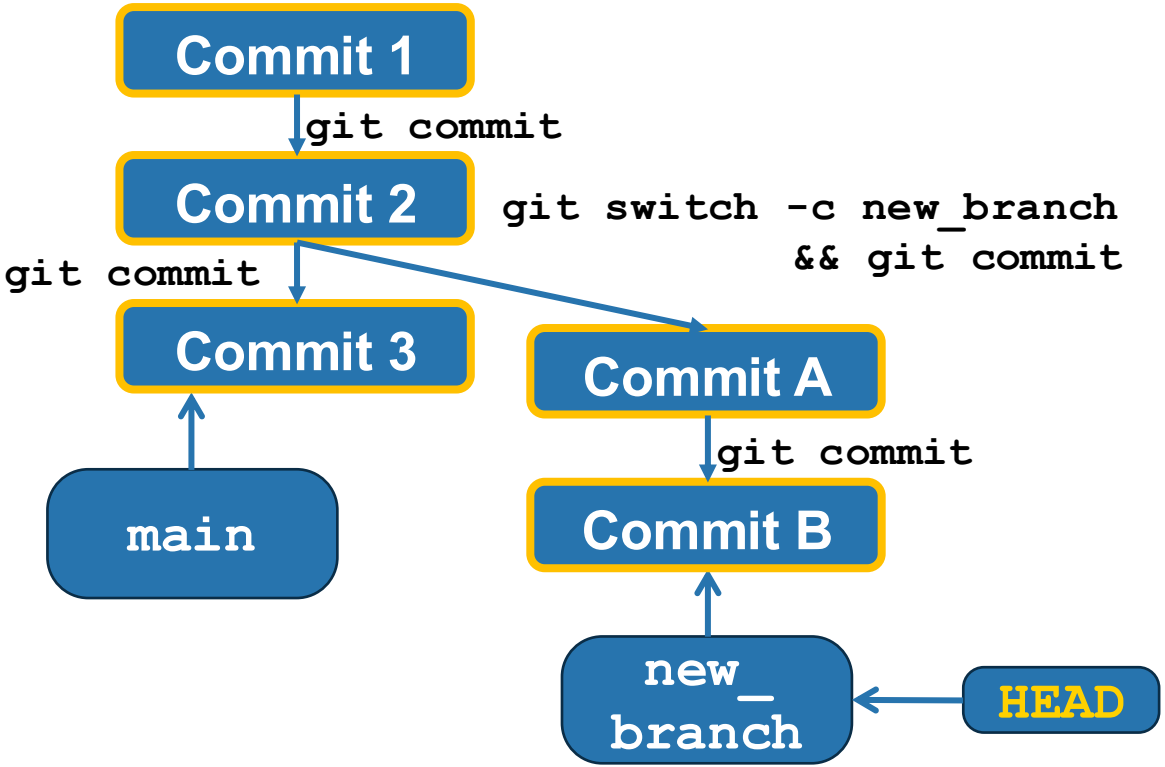
See https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Create a new branch with `git switch -c`



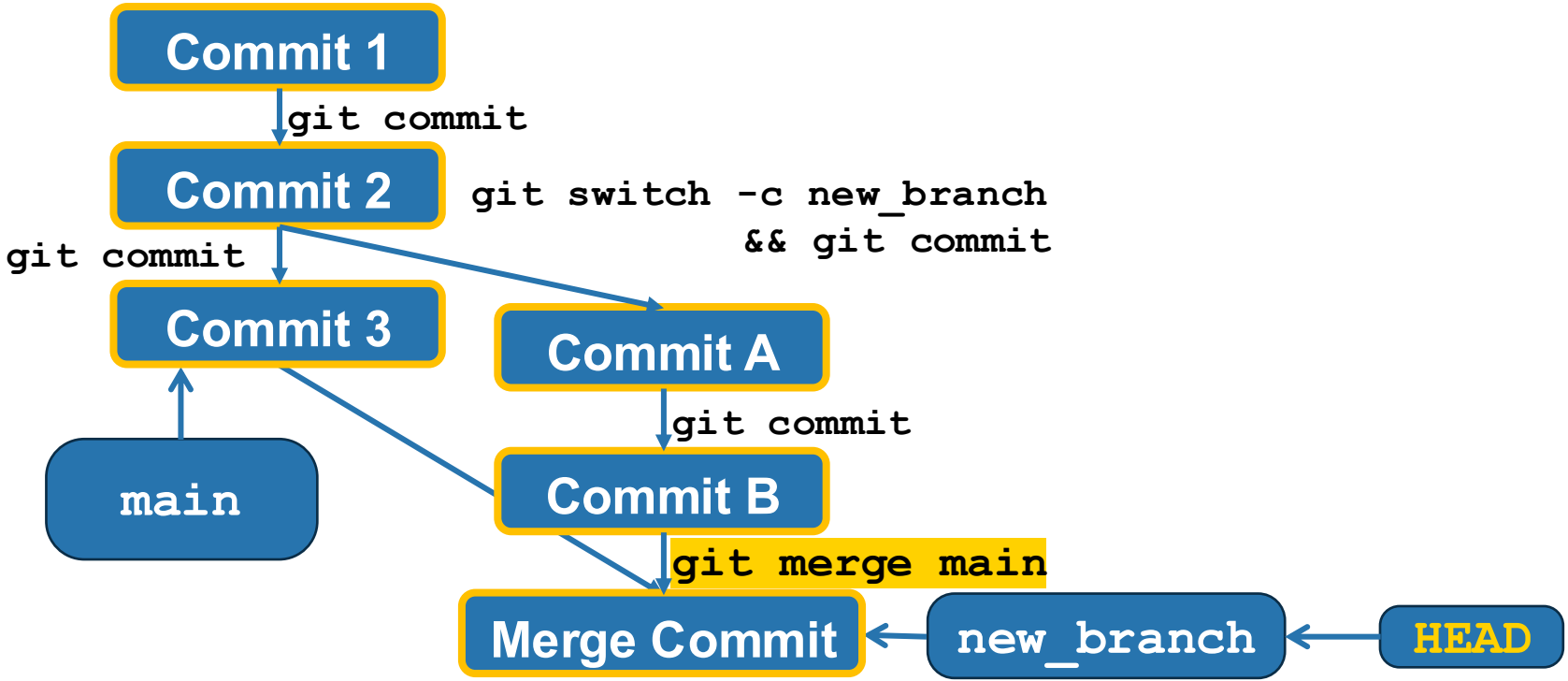
See https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Create a new branch with `git switch -c`



Combine Changes from Two Different Sources via `git merge`

See https://git-scm.com/docs/git-merge



Merge Conflicts Happen when Both Versions Changed the Same Portions of the Same File

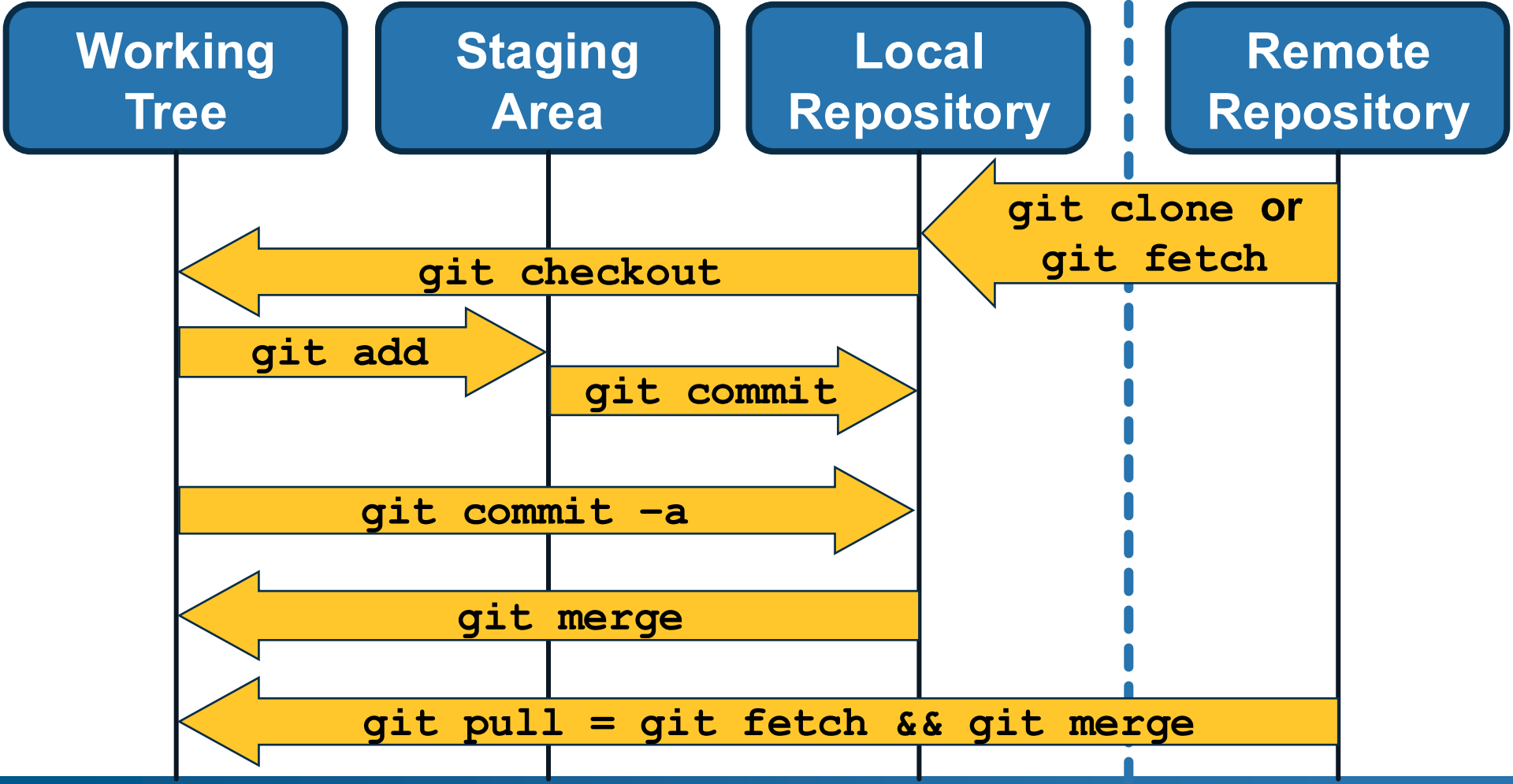
```
def format_name(first, last):
  """Prints the name"""
  return f"{first} {last}"
```

```
def format_name(first, last):
  """Prints the name"""
  return f"{last}, {first}"
```

```
def format_name(first, last):
  """Prints the name"""
  return f"first: {first} last: {last}"
```

```
def format_name(first, last):
  """Prints the name"""
<<<<<<< HEAD
  return f"{last}, {first}"
=======
  return f"first: {first} last: {last}”
>>>>>>> feature-B
```

See https://www.atlassian.com/git/tutorials/using-branches/merge-conflicts


Working Tree
Staging Area
Local Repository
Remote Repository
git clone or git fetch
git checkout
git add
git commit
git commit -a
git merge
git pull = git fetch && git merge

View the Version History via `git log`

See https://git-scm.com/docs/git-log

```
commit ec04df33efa537b3baa4535027b022ec274a1a33 (HEAD -> main, origin/main, origin/HEAD)
Author: Omar Elamri <omar@cs.ucla.edu>
Date:   Mon Apr 28 17:53:25 2025 -0700

    Fix zip submission issues

commit fcf43c521e40b2f2f0e618feca36813023a4e679
Merge: 80e1376 345f9d5
Author: Omar Elamri <ea_do@icloud.com>
Date:   Mon Apr 28 17:37:03 2025 -0700

    Merge pull request #1 from AshutoshSundresh/patch-1
    Fix link for assignment in README

commit 345f9d546a6c12fa6ad34571090ea022a327a609
Author: Ashutosh Sundresh <62063875+AshutoshSundresh@users.noreply.github.com>
Date:   Mon Apr 28 16:38:44 2025 -0700

    Fix link for assignment in README
    Without a protocol, the markdown parser will interpret this as a relative path rather
```

HEAD indicates this is the current commit

Commit Message

View the Content of a Commit via `git show`

See https://git-scm.com/docs/git-show

```
diff --git a/helper b/helper
index cb547bd..13404e2 100755
--- a/helper
+++ b/helper
@@ -62,7 +62,10 @@ zip_command() {
     "Make sure to have a complete log of how you built your solution in chorus-
     fi
-    npm run zip >/dev/null 2>&1
+    if ! npm run zip >/dev/null 2>&1; then
+        echo "Error: Could not create assign.zip."
+        exit 1
+    fi
     mv chorus-lapilli.zip assign.zip
```

This commit changes the helper file

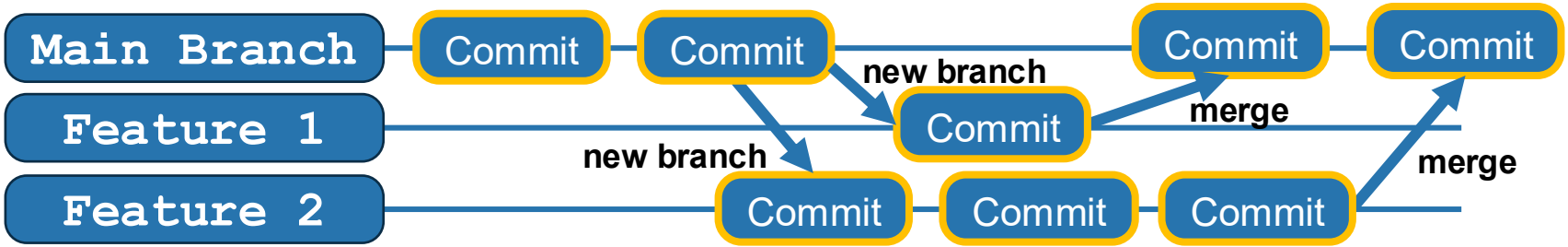
line numbers

Context

Insertions and deletions

Use Feature Branch Workflows in Team Projects!

- **Context**: **Small & coherent commits** make it easier to organize code changes.
- **Problem**: When every team member pushes their commits **to the same branch**, **incomplete features** and **frequent merge conflicts** can create confusion.
- **Solution**: Every feature gets **its own branch** that gets merged into the main branch when it adds **a stable increment to the project.**
- **Important**: **Regularly pull** from `main` to avoid **big bang merges**



Read more here: https://www.atlassian.com/git/tutorials/comparing-workflows/feature-branch-workflow

When I use `git push origin master`, I get an error:

```
git remote -v
```

Output:

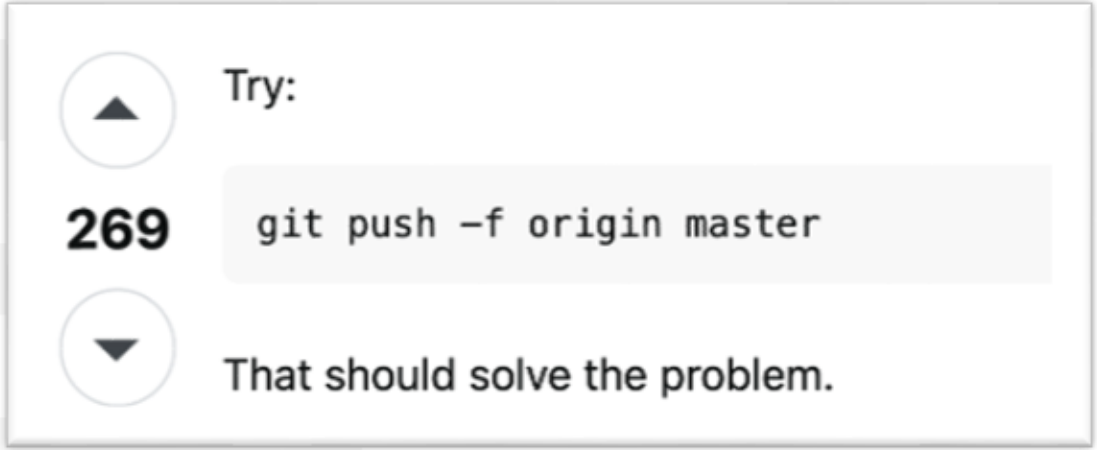
```
origin  https://github.com/REDACTED.git (fetch)
origin  https://github.com/REDACTED.git (push)
```

And:

```
git push origin master
```

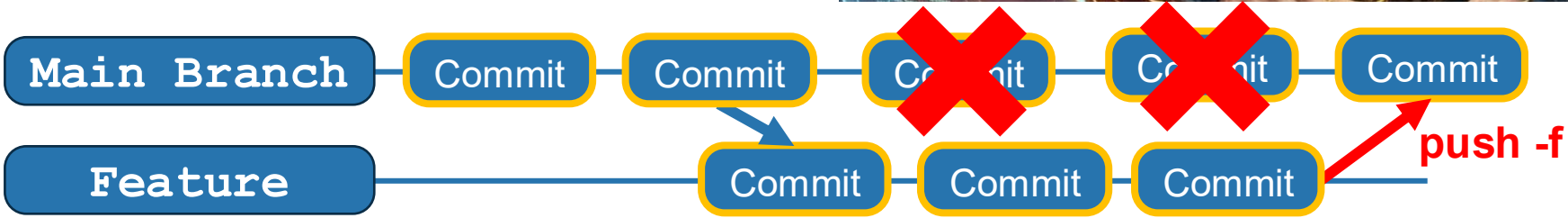
Output:

```
Username for 'https://github.com': REDACTED
Password for 'https://REDACTED@github.com':
To https://github.com/REDACTED.git
! [rejected]        master -> master (non-fast-forward)
error: failed to push some refs to 'https://github.com/REDACTED.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. Integrate the remote changes (e.g.
hint: 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```



Never force-push (`git push -f`)

- **Force push removes all other changes that might have been pushed to the remote branch**
- Someone might have pushed changes in the meantime
- **Always merge changes first**



Write Meaningful Commit Messages

Avoid commit Messages such as "`small changes`", "`bugfix`", or "`coding`"

Commit messages are intended to **help yourself and your team members understand what and why changes were made**

Code Reviews can be done via Pull Requests

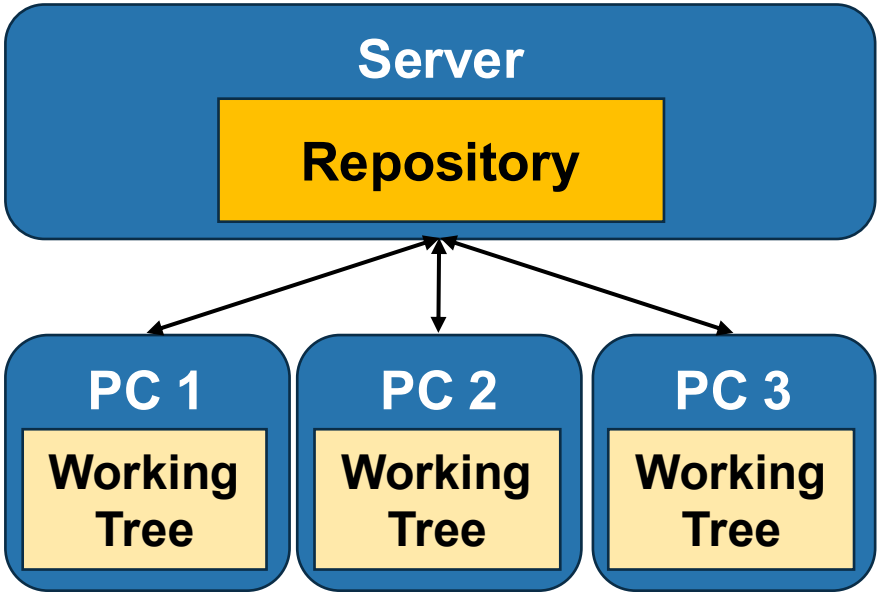
A **Pull Request (PR)** is a mechanism for a developer to notify team members that they've completed a feature or bug fix and would like to **merge** their changes into a shared repository branch (e.g., main or develop).

PR Best Practices

Keep them Small: Aim for PRs that focus on **one single, cohesive unit of work** (e.g., one feature, one bug fix). Small PRs are easier and faster to review.
Write a Clear Description: The PR title and body should clearly explain *what* the change is, *why* it was made, and any potential side effects.
Link Issues: Reference the relevant issue or ticket number (e.g., "Fixes #123").
Self-Review: Always review your own code before submitting the PR to catch obvious errors.
Respond Promptly: Be responsive to reviewer comments to keep the process moving.

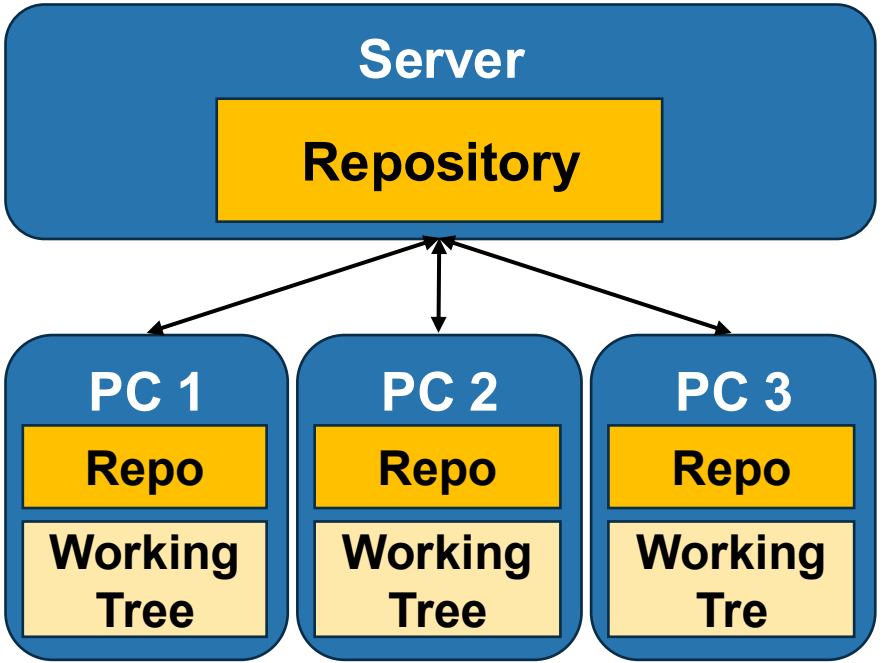
Centralized vs Distributed Version Control

Centralized Version Control



Read more here

Distributed Version Control



Centralized vs Distributed Version Control

	Centralized Version Control	Distributed Version Control
Data Storage	Data is stored in a single central repository.	Each developer has a full copy of the entire repository.
Offline Work	Requires a connection to the central server to make changes.	Developers can work offline and commit changes locally.
Use Cases	Ideal for small teams or organizations that require centralized control.	Ideal for large teams, open-source projects, and distributed teams.

Check Out the SE Book Page on Git

- https://tobiasduerschmid.github.io/SEBook/tools/git.html
- In this week's discussion you will walk through the **interactive git tutorial**
- **If you have not done so yet, please fill out the Team Formation Survey by end of today!**
  - **https://docs.google.com/forms/d/e/1FAIpQLScmPCMs6SihOnejvvBkAPXlCVlUkJyURyTMFE_u0eZdXE77YQ/viewform** Use your UCLA google account.

Please Fill out your Exit Tickets on Bruin Learn!

Question 1 1 pts

Please summarize **three insights you learned about Version Control or Git today**

Question 2 1 pts

Please describe one **reason why using Git in team projects is helpful** and one **challenge** that might arise from using Git in team projects

Question 3

Please leave any questions that you have about today's materials and things that are still unclear or confusing to you (if none, simply write N/A).

Credits: These slide use images from Flaticon.com (Creators: Freepik, pocike, Made Premium)